GUÍA 2 PARA DESARROLLAR EN PLANTUML

Relación entre casos de uso, diagrama de clases de análisis y diagramas de secuencia de análisis

Caso base: Sistema de Agendamiento de Citas

Elemento	Descripción
Asignatura	Análisis y Diseño de Software
Tema	Trazabilidad entre casos de uso, clases de análisis y secuencias de análisis con PlantUML
Caso base	Sistema de Agendamiento de Citas mediante un chatbot en el Centro Medico Renacer
Operaciones	Agendar cita en linea, Registrar paciente presencial, Gestionar disponibilidad medica y Consultar citas agendadas
Referencia base	01_Guia_PlantUML_CU_Analisis_CRUD_
Propósito	Guiar al estudiante para pasar del análisis funcional al análisis estructural y dinámico, manteniendo coherencia entre requisito, caso de uso, clase y secuencia.
Grupo / NRC	Grupo 4 - Integrantes: Jonathan Ampuero, Victor Guano, Mario Rosas
Fecha	22/10/2026

1. Propósito de la guía

Esta guía complementa la guía inicial de casos de uso. Su finalidad es mostrar cómo un CRUD de Estudiante puede analizarse de forma progresiva mediante tres modelos conectados: casos de uso de análisis, clases de análisis y diagramas de secuencia de análisis.

El énfasis no está en programar la interfaz, sino en comprender qué responsabilidad cumple cada elemento del modelo y cómo se conserva la trazabilidad desde el requisito funcional hasta la interacción entre objetos.

2. Relación general entre los tres modelos

Modelo	Pregunta que responde	Producto en PlantUML	Uso en la siguiente etapa
Casos de uso de análisis	¿Qué necesita hacer el actor con el sistema?	Diagrama de casos de uso y matriz RF-CU.	Permite identificar responsabilidades, clases frontera, control y entidad.
Clases de análisis	¿Qué objetos participan para cumplir los casos de uso?	Diagrama de clases de análisis con estereotipos boundary, control, entity y repositorio.	Define los objetos que aparecerán en los diagramas de secuencia.
Secuencias de análisis	¿Cómo colaboran actor, frontera, control y entidad para ejecutar cada caso de uso?	Diagramas de secuencia por operación del CRUD del estudiante	Valida que el flujo sea coherente con los casos de uso y las clases identificadas.

3. Caso base actualizado: AgendaMed Renacer

El caso se basa en una interfaz sencilla denominada CRUD Estudiantes, con tres campos de entrada y cuatro operaciones principales.

Dato	Tipo sugerido	Descripción	Validación mínima
Paciente	Registro de persona	Persona que solicita atención médica mediante el bot o de manera presencial.	Identificación con cédula (10 dígitos), pasaporte (8 a 9 alfanuméricos) o RUC,nombres, teléfono, correo y consentimiento.
Cita	Registro transaccional	Reserva del paciente con médico, especialidad, sucursal, fecha y hora.	Estado de la cita: PENDIENTE_PAGO, CONFIRMADA, PAGADA, ASISTIDA o CANCELADA.
Disponibilidad	Agenda / franja horaria	Horarios disponibles en los que el médico atiende en una sucursal.	Duración según especialidad dentro del horario del centro médico en donde no hay cruces de horario
Médico	Registro de personal	Profesional que atiende al paciente según su especialidad.	Especialidad, sucursal de atención y estado activo o en guardia.
Sucursal / Especialidad	Catálogo	Sedes y especialidades que dispone el centro médico.	Solo se muestran sedes y especialidades activas.

Opción / Acción de interfaz	Caso de uso asociado	Descripción de análisis
RenaBot (chat): opción "Agendar cita"	Agendar cita en línea (UC1)	El paciente elige sucursal, especialidad, médico, fecha y hora con cupos reales, confirma el resumen y recibe el código de la cita en estado PENDIENTE_PAGO.
Dashboard de recepción: Buscar / Guardar paciente	Registrar paciente presencial (UC2)	La recepcionista registra o actualiza al paciente que llega sin cita previa, valida sus datos y le asigna un turno en su sucursal.
Panel del médico: Guardar / Bloquear franja	Gestionar disponibilidad médica (UC3)	El médico registra, edita o bloquea franjas de su semana según turnos o guardias, sin chocar con citas reservadas.
Panel de agenda: Filtrar / Consultar	Consultar citas agendadas (UC4)	El médico ve su agenda del día, mientras que el administrador ve todas las citas filtrando por sucursal, especialidad, médico, fecha y estado.
RenaBot: "Hablar con un asesor", 3 errores o timeout	Derivar a recepcionista (UC7, <<extend>>)	La conversación pasa a HUMAN_HANDOFF o PENDIENTE_REVISION para que una recepcionista continúe la atención.

4. Requisitos funcionales de nivel 0

Código	Requisito funcional de nivel 0	Actor principal	Caso de uso asociado
RF-01	El sistema debe permitir al paciente agendar una cita médica en línea las 24 horas mediante el bot de mensajería, seleccionando sucursal, especialidad, médico, fecha y hora con cupos reales, y confirmando el resumen antes de registrarla.	Paciente	Agendar cita en línea (UC1)
RF-02	El sistema debe permitir a la recepcionista registrar a los pacientes que acuden de forma presencial sin cita previa, validando sus datos y asignándoles un turno en su sucursal.	Recepcionista	Registrar paciente presencial (UC2)

RF-03	El sistema debe permitir al médico registrar, actualizar o bloquear su disponibilidad semanal según sus turnos o guardias.	Médico	Gestionar disponibilidad médica (UC3)
RF-04	El sistema debe permitir consultar las citas agendadas al médico su agenda del día y al administrador todas las citas con filtros como sucursal, especialidad, médico, fecha y estado.	Médico / Administrador	Consultar citas agendadas (UC4)
RF-01	El sistema debe impedir reservar una franja ocupada o fuera del horizonte de 30 días.	Sistema	Validar disponibilidad de horario (UC5, <<include>>)
RF-02	El sistema debe impedir guardar pacientes con identificación inválida, correo inválido o sin consentimiento previo.	Sistema	Validar datos del paciente (UC6, <<include>>)
RF-01	Ante la solicitud del paciente, 3 errores o timeout, el chat debe pasar a HUMAN_HANDOFF o PENDIENTE_REVISION.	Recepcionista	Derivar a recepcionista (UC7, <<extend>>)

5. Diagrama de casos de uso de análisis

El diagrama de casos de uso representa las funciones visibles para el actor. En este caso representa las funciones visibles de los actores Paciente, Recepcionista, Médico y Administrador dentro del Centro Medico Renacer.

Código PlantUML sugerido

```
@startuml CU_Nivel0_AgendaMed
left to right direction
skinparam packageStyle rectangle
skinparam shadowing false
 actor "Paciente" as PAC
actor "Administrador" as ADM
actor "Recepcionista" as REC
actor "Médico" as MED
 rectangle "AgendaMed Renacer - Bot de agendamiento (Nivel 0)" {
  usecase "Agendar cita\nen línea" as UC1
  usecase "Registrar paciente\npresencial" as UC2
  usecase "Gestionar disponibilidad\nmédica" as UC3
  usecase "Consultar citas\nagendadas" as UC4
  usecase "Validar disponibilidad\nde horario" as UC5
  usecase "Validar datos\ndel paciente" as UC6
  usecase "Derivar a\nrecepcionista" as UC7
}
PAC -- UC1
ADM -- UC1
ADM -- UC2
ADM -- UC3
ADM -- UC4
UC1 -- REC
UC2 -- REC
UC3 -- MED
UC4 -- MED
```

```
UC1 ..> UC5 : <<include>>
UC2 ..> UC6 : <<include>>
UC7 ..> UC1 : <<extend>>
@enduml
```

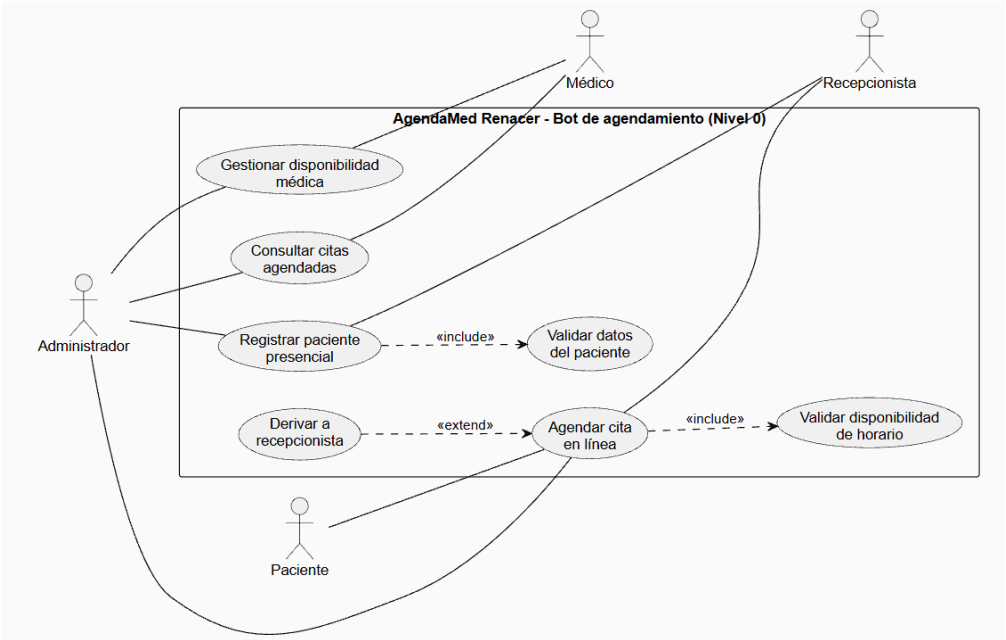


Figura 1. Diagrama de casos de uso de nivel 0

6. Derivación de clases de análisis

A partir de los casos de uso se identifican clases de análisis. Para mantener una separación clara de responsabilidades se recomienda usar los estereotipos boundary, control y entity. Además, se puede incluir un repositorio para representar la persistencia o colección de estudiantes sin entrar aún en detalles técnicos de base de datos.

Tipo de clase	Responsabilidad en el análisis	Clase propuesta	Relación con el CRUD
Boundary / Frontera	Recibe datos del actor y muestra resultados	InterfazChatBot, FormularioPacientePresencial, PanelDisponibilidad, PanelAgendaCitas	Chat de RenaBot para el paciente, dashboard de recepción, panel del médico, panel de agenda para médico y administrador
Control	Coordina la lógica de cada caso de uso	ControlAgendamiento, ControlPaciente, ControlDisponibilidad, ControlConsultaCitas	Uno por caso de uso: agendar y derivar, registrar paciente, gestionar franjas, consultar citas
Entity / Entidad	Representa la información del dominio	Cita, Paciente, Disponibilidad, Médico, Especialidad, Sucursal, Conversación, FiltroCitas, EstadoCita	Datos del SRS: paciente, cita con estado, franjas del médico, catálogos y estado de la conversación del bot
Repositorio / Colección	Conserva y recupera registros	RepositorioCita, RepositorioPaciente, RepositorioDisponibilidad, RepositorioCatalogo	Guardar y buscar citas, pacientes y franjas, listar sucursales, especialidades y médicos activos

Código PlantUML del diagrama de clases de análisis

```
@startuml Clases_Analisis_General
skinparam classAttributeIconSize 0
skinparam shadowing false
skinparam linetype ortho
hide members
hide enum members
package "Frontera (Vista)" {
  class "InterfazChatBot" <<boundary>> {
    + recibirMensaje(texto: String)
    + mostrarOpciones(opciones: List)
    + mostrarResumen(cita: Cita)
    + mostrarMensaje(mensaje: String)
  }
  class "FormularioPacientePresencial" <<boundary>> {
    - txtIdentificacion: String
    - txtNombres: String
    - txtTelefono: String
    - txtCorreo: String
    - chkConsentimiento: boolean
    + clickBuscar()
    + clickGuardar()
    + mostrarPaciente(paciente: Paciente)
    + mostrarTurno(cita: Cita)
    + mostrarMensaje(mensaje: String)
  }
  class "PanelDisponibilidad" <<boundary>> {
    + abrirAgenda()
    + clickGuardarFranja()
    + clickBloquearFranja()
    + mostrarAgendaSemanal(franjas: List<Disponibilidad>)
    + mostrarMensaje(mensaje: String)
  }
  class "PanelAgendaCitas" <<boundary>> {
    + seleccionarFiltros(filtros: FiltroCitas)
    + clickConsultar()
    + mostrarListado(citas: List<Cita>)
    + mostrarMensaje(mensaje: String)
  }
}

package "Control (Controlador)" {
  class "ControlAgendamiento" <<control>> {
    + iniciarConversacion(telefono: String): Conversacion
    + listarSucursales(): List<Sucursal>
    + listarEspecialidades(sucursalId: String): List<Especialidad>
    + listarMedicos(sucursalId: String, especialidadId: String): List<Medico>
```

```
    + consultarFranjasLibres(medicoId: String, fecha: Date): List<Disponibilidad>
    + validarDisponibilidad(medicoId: String, fecha: Date, hora: Time): boolean
    + confirmarCita(pacienteId: String, medicoId: String, fecha: Date, hora: Time): Cita
    + derivarARecepcionista(conversacionId: String, motivo: String)
  }
  class "ControlPaciente" <<control>> {
    + buscarPaciente(identificacion: String): Paciente
    + validarDatosPaciente(identificacion: String, nombres: String, telefono: String, correo: String,
consentimiento: boolean): boolean
    + registrarPacientePresencial(identificacion: String, nombres: String, telefono: String, correo:
String, consentimiento: boolean, sucursalId: String, especialidadId: String): Cita
  }
  class "ControlDisponibilidad" <<control>> {
    + obtenerAgendaSemanal(medicoId: String, semana: Date): List<Disponibilidad>
    + validarFranja(medicoId: String, dia: Date, horaInicio: Time, horaFin: Time): boolean
    + guardarFranja(medicoId: String, sucursalId: String, dia: Date, horaInicio: Time, horaFin:
Time): Disponibilidad
    + bloquearFranja(franjaId: String): boolean
  }
  class "ControlConsultaCitas" <<control>> {
    + consultarCitas(rol: String, usuarioId: String, filtros: FiltroCitas): List<Cita>
  }
}


package "Repositorio (Persistencia)" {
  class "RepositorioPaciente" <<repository>> {
    + buscarPorIdentificacion(identificacion: String): Paciente
    + buscarPorTelefono(telefono: String): Paciente
    + guardar(paciente: Paciente)
    + actualizar(paciente: Paciente)
  }
  class "RepositorioCita" <<repository>> {
    + guardar(cita: Cita)
    + buscarPorFiltros(filtros: FiltroCitas): List<Cita>
    + existenCitasEnFranja(medicoId: String, dia: Date, horaInicio: Time, horaFin: Time): boolean
  }
  class "RepositorioDisponibilidad" <<repository>> {
    + buscarFranjasLibres(medicoId: String, fecha: Date): List<Disponibilidad>
    + buscarFranja(medicoId: String, fecha: Date, hora: Time): Disponibilidad
    + buscarPrimeraFranjaLibre(sucursalId: String, especialidadId: String, fecha: Date):
Disponibilidad
    + buscarPorId(franjaId: String): Disponibilidad
    + listarPorMedico(medicoId: String, semana: Date): List<Disponibilidad>
    + existeSolapamiento(medicoId: String, dia: Date, horaInicio: Time, horaFin: Time): boolean
    + guardar(franja: Disponibilidad)
    + actualizar(franja: Disponibilidad)
  }
  class "RepositorioCatalogo" <<repository>> {
    + listarSucursalesActivas(): List<Sucursal>
```

```
    + listarEspecialidades(sucursalId: String): List<Especialidad>
    + listarMedicos(sucursalId: String, especialidadId: String): List<Medico>
  }
}

package "Entidad (Modelo)" {
  class "Paciente" <<entity>> {
    - id: String
    - tipoIdentificacion: String
    - identificacion: String
    - nombres: String
    - telefono: String
    - correo: String
    - consentimiento: boolean
    + actualizarDatos(nombres: String, telefono: String, correo: String)
  }
  class "Cita" <<entity>> {
    - codigo: String
    - fecha: Date
    - hora: Time
    - estado: EstadoCita
    - origen: String
    + getResumen(): String
  }
  class "Disponibilidad" <<entity>> {
    - id: String
    - dia: Date
    - horaInicio: Time
    - horaFin: Time
    - estado: String
    + estaLibre(): boolean
    + reservar()
    + bloquear()
  }
  class "Medico" <<entity>> {
    - id: String
    - nombres: String
    - estado: String
  }
  class "Especialidad" <<entity>> {
    - id: String
    - nombre: String
    - duracionMinutos: int
  }
  class "Sucursal" <<entity>> {
    - id: String
    - nombre: String
    - activa: boolean
```

```
  }
  class "Conversacion" <<entity>> {
    - id: String
    - telefono: String
    - estado: String
    - intentosFallidos: int
    + registrarError()
    + derivar(motivo: String)
  }
  class "FiltroCitas" <<entity>> {
    - fecha: Date
    - sucursalId: String
    - especialidadId: String
    - medicoId: String
    - estado: EstadoCita
  }
  enum "EstadoCita" {
    PENDIENTE_PAGO
    CONFIRMADA
    PAGADA
    ASISTIDA
    CANCELADA
  }
}
InterfazChatBot --> ControlAgendamiento
FormularioPacientePresencial --> ControlPaciente
PanelDisponibilidad --> ControlDisponibilidad
PanelAgendaCitas --> ControlConsultaCitas
ControlAgendamiento --> RepositorioCatalogo
ControlAgendamiento --> RepositorioDisponibilidad
ControlAgendamiento --> RepositorioCita
ControlAgendamiento --> RepositorioPaciente
ControlAgendamiento --> Conversacion
ControlPaciente --> RepositorioPaciente
ControlPaciente --> RepositorioDisponibilidad
ControlPaciente --> RepositorioCita
ControlDisponibilidad --> RepositorioDisponibilidad
ControlDisponibilidad --> RepositorioCita
ControlConsultaCitas --> RepositorioCita
Cita "0..*" --> "1" Paciente
Cita "0..*" --> "1" Medico
Cita "0..*" --> "1" Especialidad
Cita "0..*" --> "1" Sucursal
Cita "1" --> "1" Disponibilidad : ocupa
Cita --> EstadoCita
Medico "0..*" --> "1" Especialidad
Medico "0..*" --> "1" Sucursal
Disponibilidad "0..*" --> "1" Medico
```

```
Disponibilidad "0..*" --> "1" Sucursal

@enduml
```

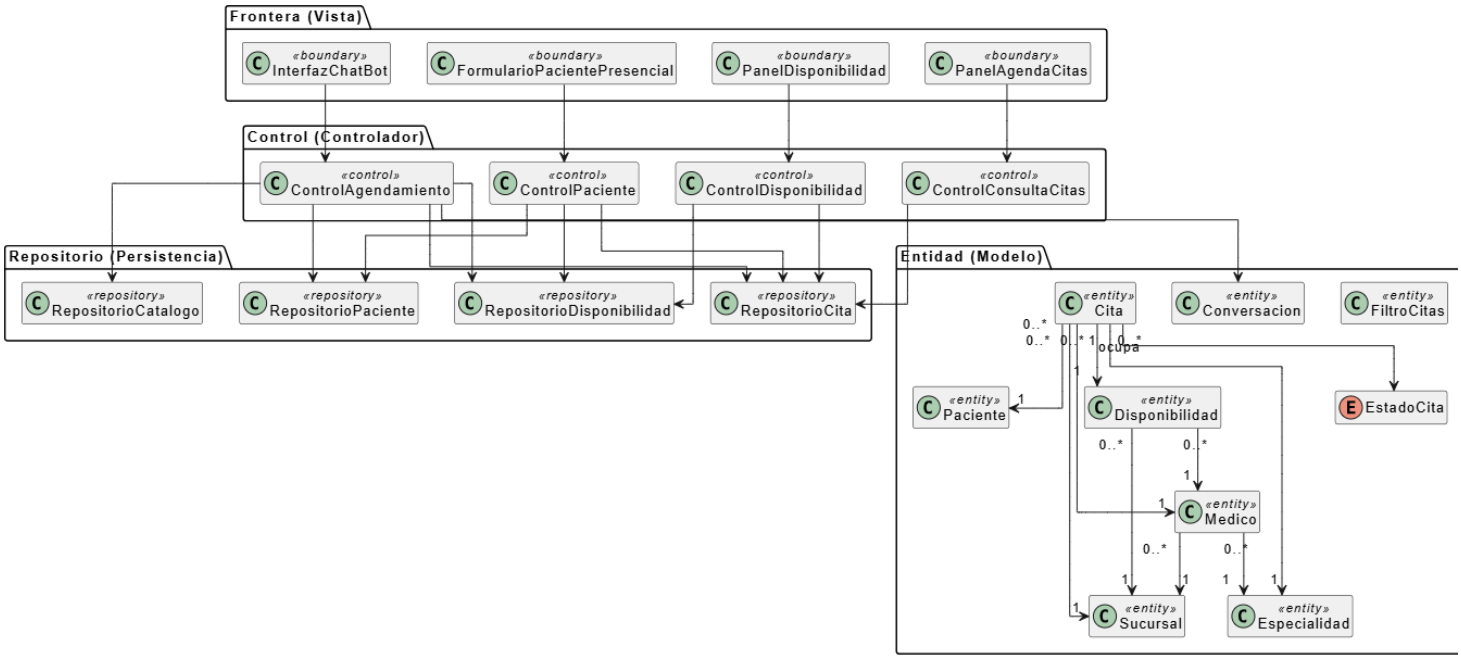


Figura 2. Vista general de clases de análisis

7. Matriz de relación: casos de uso, clases y secuencias

Caso de uso	Boundary	Control	Entity / Repositorio	Diagrama de secuencia esperado
RF-01 · Agendar cita en línea (UC1)	InterfazChatBot recibe mensajes y muestra opciones y resumen.	ControlAgendamiento: iniciarConversacion(), listar..., consultarFranjasLibres(), confirmarCita().	Crea Cita (PENDIENTE_PAGO), reserva Disponibilidad; RepositorioCita, RepositorioDisponibilidad, RepositorioCatalogo, RepositorioPaciente.	Secuencia 8.1: mensaje -> sucursal -> especialidad -> médico -> fecha/hora -> validar -> crear cita -> resumen.
RF-02 · Registrar paciente presencial (UC2)	FormularioPacientePresencial captura identificación, nombres, teléfono, correo y consentimiento.	ControlPaciente: buscarPaciente(), validarDatosPaciente(), registrarPacientePresencial().	Crea o actualiza Paciente; crea Cita (PRESENCIAL, CONFIRMADA); RepositorioPaciente, RepositorioDisponibilidad, RepositorioCita.	Secuencia 8.2: buscar -> ingresar datos -> validar -> guardar paciente -> asignar turno -> mostrar turno.
RF-03 · Gestionar disponibilidad médica (UC3)	PanelDisponibilidad muestra la agenda semanal y recibe franjas.	ControlDisponibilidad: obtenerAgendaSemanal(), validarFranja(), guardarFranja(), bloquearFranja().	Crea o bloquea Disponibilidad; RepositorioDisponibilidad y RepositorioCita (citas reservadas).	Secuencia 8.3: abrir agenda -> registrar/editar o bloquear -> validar -> guardar -> agenda actualizada.
RF-04 · Consultar citas agendadas (UC4)	PanelAgendaCitas recibe filtros y muestra el listado.	ControlConsultaCitas: consultarCitas() aplicando el rol del usuario.	RepositorioCita.buscarPorFiltros() devuelve Cita; FiltroCitas.	Secuencia 8.4: filtros -> consultar -> restringir por rol -> mostrar listado.
RF-01 · Validar disponibilidad de horario (UC5)	—	ControlAgendamiento: validarDisponibilidad().	Disponibilidad.estaLibre(); RepositorioDisponibilidad.buscarFranja().	Fragmento <<include>> dentro de la secuencia 8.1.

RF-02 · Validar datos del paciente (UC6)	—	ControlPaciente: validarDatosPaciente().	—	Fragmento <<include>> dentro de la secuencia 8.2.
RF-01 · Derivar a recepcionista (UC7)	InterfazChatBot informa al paciente.	ControlAgendamiento: derivarARecepcionista().	Conversacion.derivar() cambia el estado a HUMAN_HANDOFF / PENDIENTE_REVISION.	Fragmento opt <<extend>> dentro de la secuencia 8.1.

8. Diagramas de secuencia de análisis

Los diagramas de secuencia muestran cómo colaboran los objetos de análisis en el tiempo. Se recomienda elaborar un diagrama por cada caso de uso principal para evitar diagramas sobrecargados.

8.1 Secuencia: Agendar cita en línea (RF-01)

```
@startuml Secuencia_CU1_AgendarCitaEnLinea
title Secuencia de análisis - Agendar cita en línea (RF-01)
skinparam shadowing false
actor "Paciente" as PAC
boundary "InterfazChatBot" as UI
control "ControlAgendamiento" as Ctrl
database "RepositorioPaciente" as RPac
database "RepositorioCatalogo" as RCat
database "RepositorioDisponibilidad" as RDisp
entity "Disponibilidad" as Disp
entity "Cita" as Cit
database "RepositorioCita" as RCita
entity "Conversacion" as Conv

PAC -> UI : Envía mensaje inicial al número del centro médico
UI -> Ctrl : iniciarConversacion(telefono)
Ctrl -> RPac : buscarPorTelefono(telefono)
RPac --> Ctrl : paciente
Ctrl -> RCat : listarSucursalesActivas()
RCat --> Ctrl : sucursales
Ctrl --> UI : sucursales
UI --> PAC : mostrarOpciones(sucursales)

PAC -> UI : Selecciona sucursal
UI -> Ctrl : listarEspecialidades(sucursalId)
Ctrl -> RCat : listarEspecialidades(sucursalId)
RCat --> Ctrl : especialidades
UI --> PAC : mostrarOpciones(especialidades)

PAC -> UI : Selecciona especialidad
UI -> Ctrl : listarMedicos(sucursalId, especialidadId)
Ctrl -> RCat : listarMedicos(sucursalId, especialidadId)
RCat --> Ctrl : medicos
UI --> PAC : mostrarOpciones(medicos)
```

```
PAC -> UI : Selecciona médico y fecha
UI -> Ctrl : consultarFranjasLibres(medicoId, fecha)
Ctrl -> RDisp : buscarFranjasLibres(medicoId, fecha)
RDisp --> Ctrl : franjas
UI --> PAC : mostrarOpciones(franjas)

PAC -> UI : Selecciona hora y confirma el resumen
UI -> Ctrl : confirmarCita(pacienteId, medicoId, fecha, hora)
group <<include>> Validar disponibilidad de horario (UC5)
  Ctrl -> Ctrl : validarDisponibilidad(medicoId, fecha, hora)
  Ctrl -> RDisp : buscarFranja(medicoId, fecha, hora)
  RDisp --> Ctrl : franja
  Ctrl -> Disp : estaLibre()
  Disp --> Ctrl : true / false
end
alt franja ocupada o fuera del horizonte de 30 días
  Ctrl --> UI : null
  UI --> PAC : mostrarMensaje("Horario no disponible, elija otra hora")
else franja libre
  Ctrl -> Cit ** : Cita(paciente, medico, especialidad, sucursal, fecha, hora, PENDIENTE_PAGO)
  Ctrl -> Disp : reservar()
  Ctrl -> RDisp : actualizar(franja)
  Ctrl -> RCita : guardar(cita)
  RCita --> Ctrl : cita registrada
  Ctrl --> UI : cita
  UI -> Cit : getResumen()
  UI --> PAC : mostrarResumen(cita)\ncódigo RENACER-XXXXX e instrucciones de pago
end

opt <<extend>> Derivar a recepcionista (UC7)\n[paciente lo solicita, 3 errores o timeout]
  UI -> Ctrl : derivarARecepcionista(conversacionId, motivo)
  Ctrl -> Conv : derivar(motivo)
  note right of Conv : estado = HUMAN_HANDOFF\no PENDIENTE_REVISION
  UI --> PAC : mostrarMensaje("Una recepcionista continuará la atención")
end
@enduml
```

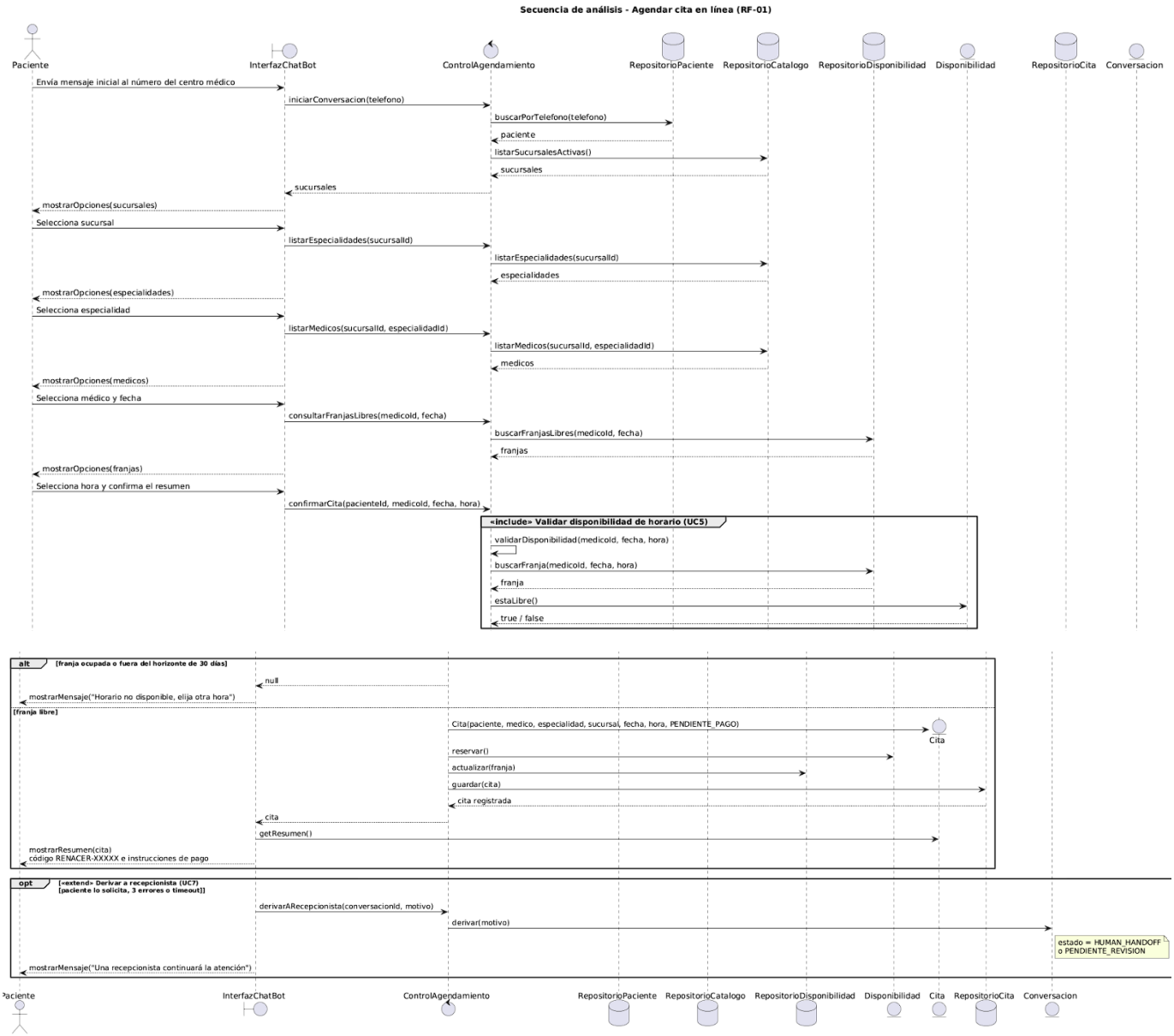


Figura 3. Secuencia de análisis - Agendar cita en línea.

8.2 Secuencia: Registrar paciente presencial (RF-02)

```
@startuml Secuencia_CU2_RegistrarPacientePresencial
title Secuencia de análisis - Registrar paciente presencial (RF-02)
skinparam shadowing false
actor "Recepcionista" as REC
boundary "FormularioPacientePresencial" as UI
control "ControlPaciente" as Ctrl
entity "Paciente" as Pac
database "RepositorioPaciente" as RPac
database "RepositorioDisponibilidad" as RDisp
entity "Disponibilidad" as Disp
entity "Cita" as Cit
database "RepositorioCita" as RCita
```

```
REC -> UI : Ingresa identificación del paciente
REC -> UI : clickBuscar()
UI -> Ctrl : buscarPaciente(identificacion)
Ctrl -> RPac : buscarPorIdentificacion(identificacion)
RPac --> Ctrl : paciente / null
Ctrl --> UI : paciente / null
UI --> REC : mostrarPaciente(paciente) o formulario vacío

REC -> UI : Ingresa o actualiza nombres, teléfono,\ncorreo y consentimiento
REC -> UI : clickGuardar()
UI -> Ctrl : registrarPacientePresencial(identificacion, nombres, telefono,\ncorreo, consentimiento,
sucursalId, especialidadId)
group <<include>> Validar datos del paciente (UC6)
  Ctrl -> Ctrl : validarDatosPaciente(identificacion, nombres,\ntelefono, correo, consentimiento)
end
alt identificación, correo inválidos o sin consentimiento
  Ctrl --> UI : null
  UI --> REC : mostrarMensaje("Datos del paciente inválidos")
else datos válidos
  alt paciente nuevo
    Ctrl -> Pac ** : Paciente(identificacion, nombres, telefono, correo, consentimiento)
    Ctrl -> RPac : guardar(paciente)
  else paciente existente
    Ctrl -> Pac : actualizarDatos(nombres, telefono, correo)
    Ctrl -> RPac : actualizar(paciente)
  end
  Ctrl -> RDisp : buscarPrimeraFranjaLibre(sucursalId, especialidadId, hoy)
  RDisp --> Ctrl : franja
  Ctrl -> Disp : reservar()
  Ctrl -> RDisp : actualizar(franja)
  Ctrl -> Cit ** : Cita(paciente, franja, origen = PRESENCIAL, CONFIRMADA)
  Ctrl -> RCita : guardar(cita)
  RCita --> Ctrl : turno registrado
  Ctrl --> UI : cita (turno)
  UI --> REC : mostrarTurno(cita)
end
@enduml
```

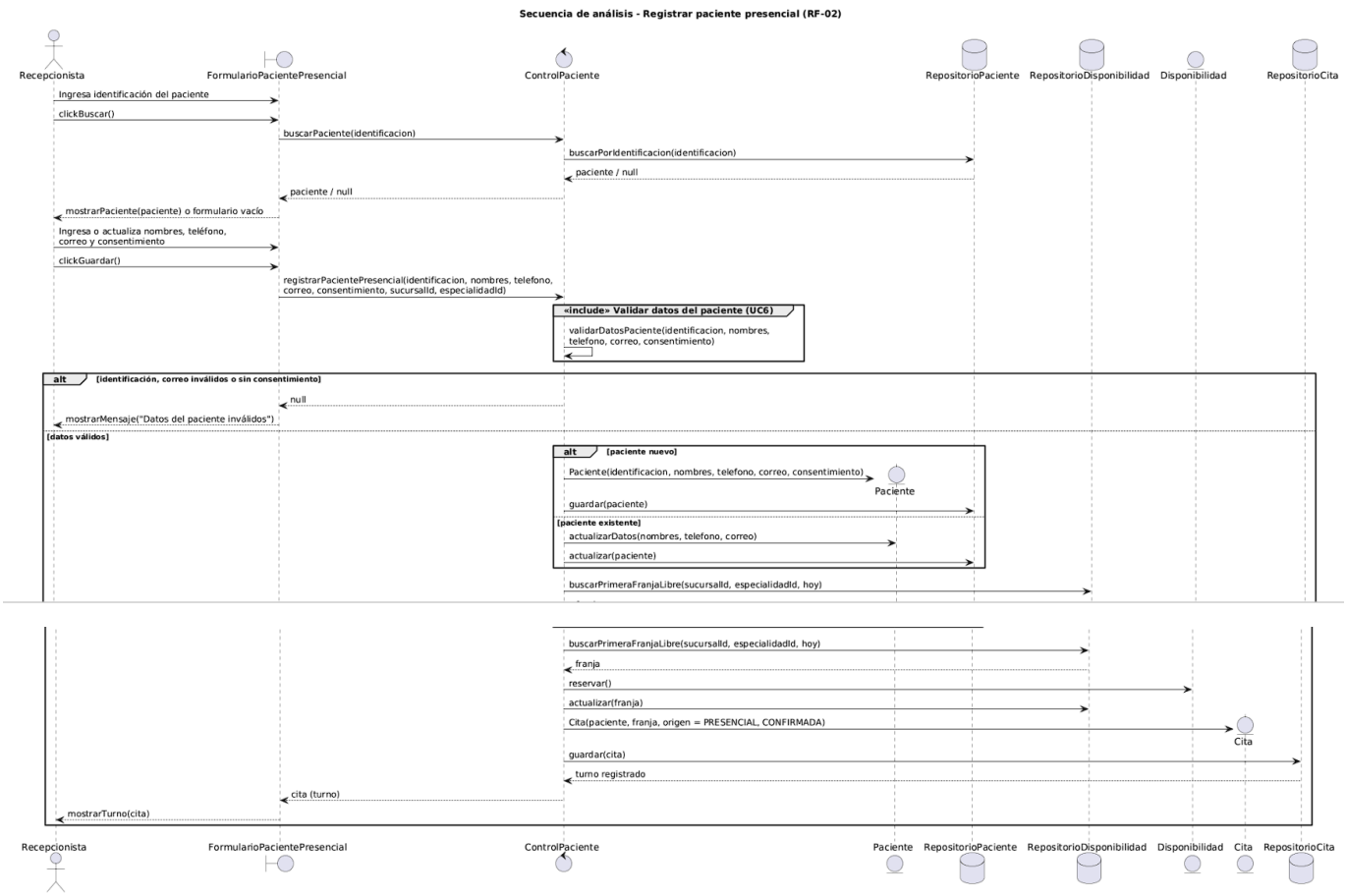


Figura 4. Secuencia de análisis - Registrar paciente presencial.

8.3 Secuencia: Gestionar disponibilidad médica (RF-03)

```
@startuml Secuencia_CU3_GestionarDisponibilidadMedica
title Secuencia de análisis - Gestionar disponibilidad médica (RF-03)
skinparam shadowing false
actor "Médico" as MED
boundary "PanelDisponibilidad" as UI
control "ControlDisponibilidad" as Ctrl
database "RepositorioDisponibilidad" as RDisp
database "RepositorioCita" as RCita
entity "Disponibilidad" as Disp

MED -> UI : abrirAgenda()
UI -> Ctrl : obtenerAgendaSemanal(medicoId, semana)
Ctrl -> RDisp : listarPorMedico(medicoId, semana)
RDisp --> Ctrl : franjas
Ctrl --> UI : franjas
UI --> MED : mostrarAgendaSemanal(franjas)

alt registrar o editar franja
  MED -> UI : Ingresa día, hora inicio y hora fin
  MED -> UI : clickGuardarFranja()
```

```
  UI -> Ctrl : guardarFranja(medicoId, sucursalId, dia, horaInicio, horaFin)
  Ctrl -> Ctrl : validarFranja(medicoId, dia, horaInicio, horaFin)
  Ctrl -> RDisp : existeSolapamiento(medicoId, dia, horaInicio, horaFin)
  RDisp --> Ctrl : true / false
  Ctrl -> RCita : existenCitasEnFranja(medicoId, dia, horaInicio, horaFin)
  RCita --> Ctrl : true / false
  alt fuera del horario del centro, solapada o con citas reservadas
    Ctrl --> UI : null
    UI --> MED : mostrarMensaje("La franja no es válida")
  else franja válida
    Ctrl -> Disp ** : Disponibilidad(medico, sucursal, dia, horaInicio, horaFin, LIBRE)
    Ctrl -> RDisp : guardar(franja)
    Ctrl --> UI : franja
    UI --> MED : mostrarAgendaSemanal(franjas)
  end
else bloquear franja (turno o guardia)
  MED -> UI : clickBloquearFranja()
  UI -> Ctrl : bloquearFranja(franjaId)
  Ctrl -> RDisp : buscarPorId(franjaId)
  RDisp --> Ctrl : franja
  Ctrl -> Disp : estaLibre()
  Disp --> Ctrl : true / false
  alt franja con cita reservada
    Ctrl --> UI : false
    UI --> MED : mostrarMensaje("La franja tiene citas reservadas")
  else franja libre
    Ctrl -> Disp : bloquear()
    Ctrl -> RDisp : actualizar(franja)
    Ctrl --> UI : true
    UI --> MED : mostrarAgendaSemanal(franjas)
  end
end
@enduml
```

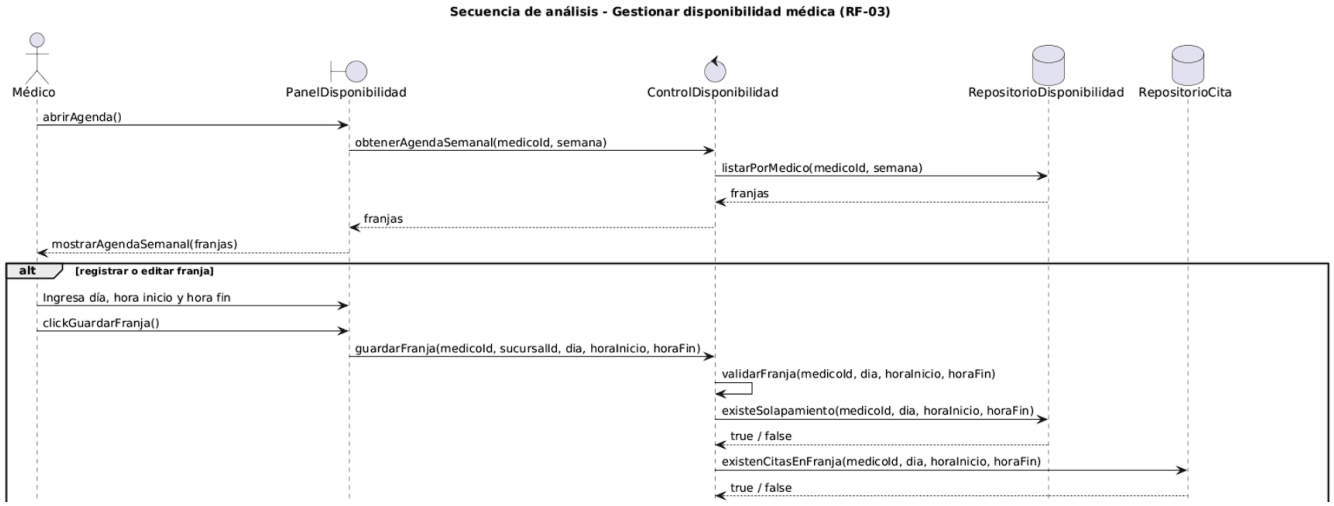




Figura 5. Secuencia de análisis - Gestionar disponibilidad médica.

8.4 Secuencia: Consultar citas agendadas (RF-04)

```
@startuml Secuencia_CU4_ConsultarCitasAgendadas
title Secuencia de análisis - Consultar citas agendadas (RF-04)
skinparam shadowing false
actor "Médico / Administrador" as USR
boundary "PanelAgendaCitas" as UI
control "ControlConsultaCitas" as Ctrl
database "RepositorioCita" as RCita
entity "Cita" as Cit

USR -> UI : seleccionarFiltros(filtros)\nfecha, sucursal, especialidad, médico, estado
USR -> UI : clickConsultar()
UI -> Ctrl : consultarCitas(rol, usuarioId, filtros)
note over Ctrl
  Si rol = MÉDICO: filtros.medicoId = usuarioId
  (el médico solo ve su agenda).
  Si rol = ADMINISTRADOR: se aplican todos
  los filtros seleccionados.
end note
Ctrl -> RCita : buscarPorFiltros(filtros)
RCita --> Ctrl : listaCitas
alt sin resultados
  Ctrl --> UI : lista vacía
  UI --> USR : mostrarMensaje("No existen citas con esos filtros")
else existen citas
  loop por cada cita
    Ctrl -> Cit : getResumen()
```

```
    Cit --> Ctrl : paciente, hora, sucursal y estado
  end
  Ctrl --> UI : listaCitas
  UI --> USR : mostrarListado(citas)
end
@enduml
```

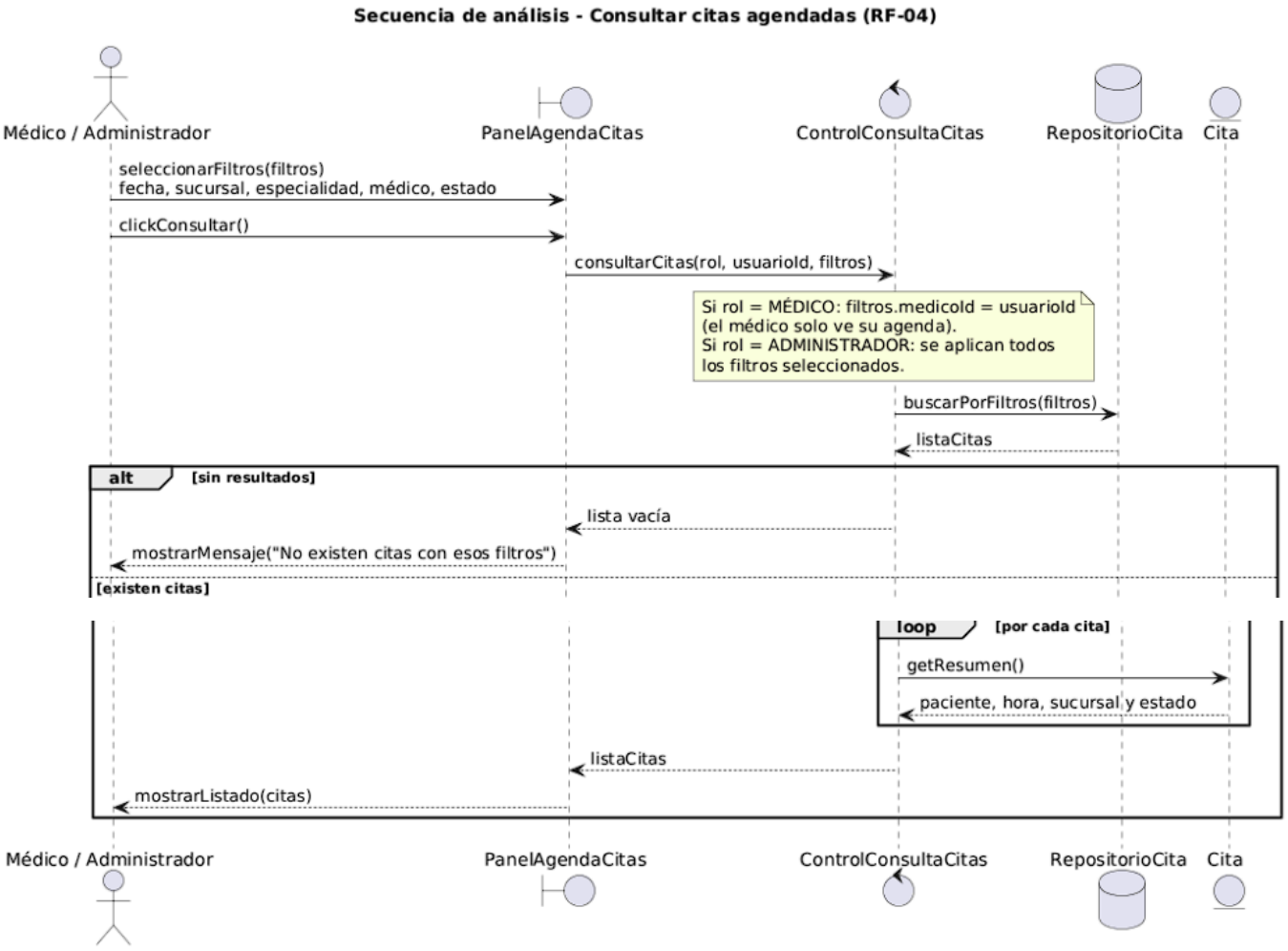


Figura 6. Secuencia de análisis - Consultar citas agendadas.

9. Especificación breve de casos de uso conectada con las secuencias

Caso de uso	Precondición	Flujo principal resumido	Resultado esperado
Agendar cita en línea (Actor: Paciente)	El paciente escribe al número del centro médico y está registrado; existen sucursales y médicos con disponibilidad.	Enviar mensaje inicial -> seleccionar sucursal -> seleccionar especialidad -> seleccionar médico y fecha -> seleccionar hora y confirmar resumen -> validar disponibilidad (<<include>>) -> crear cita. Alterno: franja ocupada -> elegir otra hora; solicitud de asesor, 3 errores o timeout -> derivar a recepcionista (<<extend>>). (Secuencia 8.1)	Cita registrada con código único en estado PENDIENTE_PAGO y franja bloqueada para otros pacientes.
Registrar paciente presencial (Actor: Recepcionista)	Recepcionista autenticada en el dashboard de su sucursal; paciente sin cita previa.	Buscar paciente -> ingresar o actualizar datos -> validar datos (<<include>>) -> guardar paciente -> asignar turno -> mostrar turno.	Paciente registrado y turno presencial asignado, visible en la agenda del médico.

		Alterno: datos inválidos -> mostrar mensaje. (Secuencia 8.2)	
Gestionar disponibilidad médica (Actor: Médico)	Médico autenticado en su panel; se conoce el horario del centro y las citas reservadas.	Abrir agenda -> registrar, editar o bloquear franjas -> validar horario, solapamientos y citas reservadas -> guardar. Alterno: franja inválida o con citas -> mostrar mensaje. (Secuencia 8.3)	Disponibilidad actualizada; el bot mostrará solo cupos con franja disponible.
Consultar citas agendadas (Actor: Médico / Administrador)	Usuario autenticado; existen citas registradas.	Seleccionar fecha y filtros -> consultar citas (restringidas por rol) -> mostrar listado. Alterno: sin resultados -> mostrar mensaje. (Secuencia 8.4)	Agenda visible en tiempo real: el médico ve solo sus citas y el administrador ve todas.

10. Procedimiento recomendado para el estudiante

- Revisar la interfaz CRUD e identificar campos de datos: ID, Nombre y Edad.
- Redactar requisitos funcionales de nivel 0 para Agregar, Actualizar, Eliminar y Mostrar Todo.
- Modelar los casos de uso en PlantUML y justificar únicamente las relaciones include necesarias.
- Derivar clases de análisis: una clase frontera para la interfaz, una clase control para coordinar la lógica, una entidad Estudiante y un repositorio o colección.
- Construir un diagrama de secuencia por cada caso de uso principal.
- Verificar que cada mensaje del diagrama de secuencia corresponda a una operación declarada en el diagrama de clases.
- Actualizar la matriz de trazabilidad para demostrar coherencia entre RF, CU, clases y secuencias.

11. Lista de verificación de coherencia

- Cada requisito funcional tiene al menos un caso de uso asociado.
- Cada caso de uso principal posee una secuencia de análisis propia.
- Las clases usadas en las secuencias existen en el diagrama de clases de análisis.
- Los mensajes de secuencia coinciden con operaciones de las clases de análisis.
- La entidad Estudiante conserva los atributos ID, Nombre y Edad.
- La validación de datos se aplica antes de agregar y actualizar.
- El diagrama no mezcla detalles de implementación de interfaz gráfica con responsabilidades de análisis.

12. Rúbrica breve de evaluación sobre 20 puntos

Criterio	Descripción	Puntaje	Evidencia
Trazabilidad entre modelos	Relaciona correctamente RF, casos de uso, clases y secuencias.	5 pts	Matriz de relación completa.
Diagrama de casos de uso	Representa actor, sistema, casos de uso y relaciones include justificadas.	4 pts	Código y diagrama PlantUML.
Diagrama de clases de análisis	Identifica frontera, control, entidad y repositorio con responsabilidades claras.	4 pts	Código PlantUML de clases.
Diagramas de secuencia	Modela flujos coherentes para Agregar, Actualizar, Eliminar y Mostrar Todo.	5 pts	Cuatro secuencias PlantUML.
Presentación técnica	Documento ordenado, lenguaje claro y sin errores de sintaxis PlantUML.	2 pts	Entrega final documentada.

13. Formato mínimo de entrega sugerido

Sección	Contenido mínimo	Evidencia
Portada	Datos institucionales, asignatura, NRC, estudiante/equipo, proyecto y fecha.	Documento entregado.
Casos de uso	Requisitos funcionales, diagrama CU y especificación breve.	Código PlantUML + captura.
Clases de análisis	Diagrama con boundary, control, entity y repositorio.	Código PlantUML + captura.
Secuencias de análisis	Una secuencia por cada operación CRUD principal.	Códigos PlantUML + capturas.
Trazabilidad	Matriz que conecte RF, CU, clases y secuencias.	Tabla de relación.
Conclusión técnica	Reflexión breve sobre la coherencia entre modelos.	Texto de cierre.

14. Conclusión técnica

La relación entre casos de uso, clases de análisis y diagramas de secuencia permite comprobar que el modelo de AgendaMed Renacer no se construye como piezas aisladas ya que, los casos de uso explican qué necesitan el Paciente, la Recepcionista, el Médico y el Administrador, las clases de análisis explican qué objetos colaboran (vistas, controladores, entidades y repositorios) y las secuencias muestran cómo esos objetos interactúan para agendar citas, registrar pacientes, gestionar la disponibilidad y consultar la agenda. Esta trazabilidad desde el SRS hasta la interacción entre objetos mejora la claridad del análisis y prepara el diseño posterior del sistema.

15. Fuente interna de referencia

- 01_Guia_PlantUML_CU_Analisis_CRUD_Estudiante.
- Guía Version2 del CRUD Estudiante con campos ID, Nombre y Edad, adaptada a la interfaz de referencia proporcionada por la docente.